END-TO-END ENGINEERING IMPLEMENTATION MANUAL

Nueva Ecija Travel Itinerary Application: Step-by-Step Build Guide

Complete Zero-to-MVP Implementation Playbook for Ionic Angular + Capacitor, Python FastAPI, PostgreSQL, and Leaflet + OpenStreetMap

This implementation guide translates the **Nueva Ecija Travel Itinerary MVP Development Plan** into concrete, sequential engineering steps. Every command, schema definition, API contract, recommendation scoring formula, and frontend lifecycle hook is designed so you can build, verify, and pilot the application from scratch on a single development laptop without relying on paid Google Maps, paid Places, paid routing, or paid generative-AI APIs.

Build Roadmap at a Glance (11 Sequential Milestones)

Milestone	Engineering Focus	Exit Gate Verification
Step 0	Monorepo & Scope Freeze	Clean Git tree, `.gitignore`, and `.env.example` with zero hardcoded secrets.
Step 1	PostgreSQL & FastAPI Core	7 SQLAlchemy 2.0 tables migrated via Alembic; `GET /api/v1/health` returns `200 OK`.
Step 2	OSM POI Import Pipeline	Idempotent Overpass import script & curated Nueva Ecija featured destinations seeded.
Step 3	Auth & Ionic App Shell	Argon2id Email Auth, server-verified Google OAuth, local Guest Mode, and 4-tab shell.
Step 4	Home & Debounced Search	Local PostgreSQL search with 300ms RxJS debounce and curated Popular Destinations.
Step 5	Leaflet Map & 16 Filters	Bounded Nueva Ecija map, OSM attribution, marker bottom sheet, and external Maps URLs.
Step 6	Custom Itinerary Builder	BOLA-checked CRUD, custom stops, reordering, honest cost math, and dirty-form guards.
Step 7	Suggested Itinerary Engine	Deterministic weighted scoring, Haversine clustering, and editable draft generation.
Step 8	Overview, Day View & Sync	Day selector chips, stop detail cards, Home persistence, and optional guest migration.
Step 9	Pilot Security & Tunneling	Local Wi-Fi / Cloudflare Tunnel access, rate limiting, and tested `pg_dump` restore.
Step 10	QA & Pilot Acceptance	All automated unit/API tests and the 24-point MVP QA checklist pass.

Geographic Scope	Client Stack	Backend & Data Stack	Blueprint Alignment
Province of Nueva Ecija, Central Luzon, Philippines	Ionic Angular + Capacitor, Leaflet, Ionic Storage	Python 3.11+ FastAPI, SQLAlchemy 2.0, Alembic, PostgreSQL 15+	27 October 2026 Product & Engineering Blueprint

1. Architecture Overview & Free-Resource Guardrails

The Nueva Ecija Travel Itinerary MVP enforces strict separation between the mobile client, REST API, and database while honoring all open-data usage policies. Before writing code, lock in the architectural rules below so you never have to retrofit security boundaries or rip out policy-violating third-party calls later.

1.1 High-Level Component Topology & Data Flow

- **Ionic Angular + Capacitor Client (`mobile/`)**: Renders the Welcome/Login/Signup/Guest entry flow and the four-tab navigation shell (`Home`, `Map`, and explicit `"Planned for a later release"` placeholders for `History` and `Profile`). Authenticated users read and write itineraries via FastAPI over HTTPS REST/JSON; Guest users store itineraries locally on-device using `@ionic/storage-angular`.
- **Python FastAPI Backend (`backend/`)**: Exposes versioned `/api/v1` endpoints, hashes passwords with Argon2id, verifies Google ID tokens server-side, executes local place search and bounding-box filtering, runs the deterministic suggested-itinerary scoring engine, and enforces strict object ownership checks on every itinerary mutation.
- **PostgreSQL Database (`127.0.0.1:5432`)**: Persists `users`, `auth_identities`, `municipalities`, `places`, `itineraries`, `itinerary_days`, and `itinerary_items`. The Angular app never connects directly to PostgreSQL, and port `5432` is never exposed to client devices or the internet.
- **Offline POI Import Pipeline (`scripts/`)**: Processes a one-time or scheduled OpenStreetMap / Overpass export for Nueva Ecija, validates coordinate bounds, maps tags to 16 canonical categories, and upserts records into PostgreSQL with full source traceability.

1.2 Non-Negotiable Zero-Cost & Open-Data Compliance Matrix

Integration Area	Implementation Requirement	Authoritative Policy
Map Raster Tiles	Configure Leaflet with an environment-driven tile URL (defaulting to standard OSM raster tiles for dev/small pilot), display `© OpenStreetMap contributors` on every map view, respect HTTP cache headers, and never bulk-download offline tiles.	OSM Tile Usage Policy
Place Search & Autocomplete	Never query public Nominatim from client search inputs (client-side autocomplete is prohibited). Query your local FastAPI `GET /api/v1/places` endpoint with a 300ms debounce.	Nominatim Usage Policy & ODbL FAQ
POI Extraction	Run Overpass queries only during offline data seeding via `scripts/import_osm.py`. Never query Overpass live when users pan or open the Map tab.	Overpass API Documentation
Apple Sign-In Caveat	Apple Developer Program membership costs US$99/year. Gate Apple login behind `ENABLE_APPLE_AUTH=false` and display a truthful disabled/unavailable state until paid credentials exist.	Apple Program Enrollment

Integration Area	Implementation Requirement	Authoritative Policy
Google OAuth 2.0	Verify Google ID tokens on the FastAPI backend (checking signature, `iss`, `aud`, and `exp`) and link accounts via the immutable `sub` identifier rather than display name.	Google Sign-In Web Guide
External Directions	Hand off turn-by-turn navigation to an external browser or maps app using standard Google Maps URLs (no API key required).	Google Maps URLs Guide
Remote Pilot Access	Use an outbound Cloudflare Tunnel (`cloudflared`) to expose local port `8000` over HTTPS without opening inbound router ports.	Cloudflare Tunnel Docs

2. Step 0 — Freeze Scope, Install Prerequisites & Initialize the Monorepo

Start by installing the required open-source runtimes on your development laptop and establishing a clean single-repository structure with strict secret hygiene.

2.1 Verify Local Runtimes

- **Node.js & npm**: Install Node.js 20 LTS or 22 LTS and verify with `node -v && npm -v`.
- **Ionic & Capacitor CLI**: Run `npm install -g @ionic/cli` and verify with `ionic -v`.
- **Python 3.11+**: Verify with `python3 --version` and `python3 -m venv --help`.
- **PostgreSQL 15+**: Verify local server status with `psql --version` and `pg_isready -h 127.0.0.1`.

2.2 Scaffold the Monorepo Directory Tree

```
mkdir -p nueva-ecija-
travel/backend/app/{api/v1,core,models,schemas,services,repositories,algorithms}
mkdir -p nueva-ecija-travel/{backend/tests,data/{raw,staging,curated},scripts,docs}
cd nueva-ecija-travel
git init
```

2.3 Create `.gitignore` and `.env.example`

Create `.gitignore` at the repository root to prevent credentials, local virtual environments, build outputs, raw OSM extracts, and database backups from entering version control:

```
# Secrets and environment files
.env
*.env
*.pem
*.key
client_secret*.json

# Python artifacts
__pycache__/
*.py[cod]
.venv/
venv/
.pytest_cache/
```

```
.mypy_cache/
.ruff_cache/

# Ionic Angular & Capacitor artifacts
mobile/node_modules/
mobile/www/
mobile/.angular/
mobile/android/app/build/
mobile/ios/App/Pods/

# Data extracts & database backups
data/raw/*
!data/raw/.gitkeep
*.dump
*.sql.gz
```

Create `.env.example` with documented variable names and non-secret placeholder values, then copy it to `.env` (`cp .env.example .env`) and generate a real 64-character hex secret using `openssl rand -hex 32`:

```
APP_ENV=development
API_V1_PREFIX=/api/v1
DATABASE_URL=postgresql+psycopg://ne_travel_user:change_me_local_only@127.0.0.1:5432/nueva_ec
ija_travel
JWT_SECRET_KEY=replace_with_output_of_openssl_rand_hex_32
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30
REFRESH_TOKEN_EXPIRE_DAYS=7
CORS_ORIGINS=["http://localhost:8100","capacitor://localhost","http://localhost"]
GOOGLE_OAUTH_CLIENT_ID=your-google-client-id.apps.googleusercontent.com
ENABLE_APPLE_AUTH=false
DEFAULT_DEST_RADIUS_KM=25.0
```

Step 0 Exit Check: Run `git status` and confirm that `.env` is ignored and the directory layout matches Section 11 of the MVP blueprint.

3. Step 1 — Set Up PostgreSQL, SQLAlchemy Models & FastAPI Foundation

In Step 1, provision the local PostgreSQL database, configure SQLAlchemy 2.0 and Alembic migrations for all seven required tables, and verify backend connectivity with automated tests.

3.1 Create the Local PostgreSQL Role and Database

Connect to your local PostgreSQL instance and create a dedicated, least-privilege application role (never connect the FastAPI app as the `postgres` superuser):

```
psql -U postgres -h 127.0.0.1 << 'SQL'
CREATE ROLE ne_travel_user WITH LOGIN PASSWORD 'change_me_local_only';
CREATE DATABASE nueva_ecija_travel OWNER ne_travel_user;
GRANT ALL PRIVILEGES ON DATABASE nueva_ecija_travel TO ne_travel_user;
SQL
```

3.2 Create the Python Virtual Environment & Install Backend Packages

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
```

```
pip install --upgrade pip
pip install fastapi "uvicorn[standard]" sqlalchemy "psycopg[binary]" alembic \
  pydantic pydantic-settings "email-validator" "pwdlib[argon2]" pyjwt \
  google-auth requests httpx slowapi pytest ruff mypy
```

3.3 Implement Settings and Database Session Management

Write `backend/app/core/config.py` to load validated settings from `.env`:

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    app_env: str = "development"
    api_v1_prefix: str = "/api/v1"
    database_url: str
    jwt_secret_key: str
    jwt_algorithm: str = "HS256"
    access_token_expire_minutes: int = 30
    refresh_token_expire_days: int = 7
    cors_origins: list[str] = ["http://localhost:8100", "capacitor://localhost",
"http://localhost"]
    google_oauth_client_id: str = ""
    enable_apple_auth: bool = False
    default_dest_radius_km: float = 25.0

    model_config = SettingsConfigDict(env_file="../.env", extra="ignore")

settings = Settings()
```

Write `backend/app/core/database.py` to configure the SQLAlchemy engine and request-scoped session dependency:

```
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker
from app.core.config import settings

engine = create_engine(settings.database_url, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)

class Base(DeclarativeBase):
    pass

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

3.4 Implement the 7 Relational Models (`backend/app/models/entities.py`)

Create `backend/app/models/entities.py` with UUID primary keys, UTC timestamps, Philippine calendar dates for itineraries, and database-enforced uniqueness constraints:

```
import uuid
from datetime import date, datetime, time, timezone
from sqlalchemy import (
    Boolean, Date, DateTime, Float, ForeignKey, Index, Integer,
    Numeric, String, Text, Time, UniqueConstraint
```

```
)
from sqlalchemy.dialects.postgresql import JSONB, UUID
from sqlalchemy.orm import Mapped, mapped_column, relationship
from app.core.database import Base

def utcnow() -> datetime:
    return datetime.now(timezone.utc)

class User(Base):
    __tablename__ = "users"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    first_name: Mapped[str] = mapped_column(String(80), nullable=False)
    last_name: Mapped[str] = mapped_column(String(80), nullable=False)
    username_normalized: Mapped[str] = mapped_column(String(50), unique=True, index=True,
nullable=False)
    email_normalized: Mapped[str] = mapped_column(String(255), unique=True, index=True,
nullable=False)
    password_hash: Mapped[str | None] = mapped_column(String(255), nullable=True)
    is_active: Mapped[bool] = mapped_column(Boolean, default=True, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow,
onupdate=utcnow)

class AuthIdentity(Base):
    __tablename__ = "auth_identities"
    __table_args__ = (UniqueConstraint("provider", "provider_subject",
name="uq_provider_subject"),)
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    user_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), ForeignKey("users.id",
ondelete="CASCADE"), nullable=False)
    provider: Mapped[str] = mapped_column(String(32), nullable=False)
    provider_subject: Mapped[str] = mapped_column(String(255), nullable=False)
    provider_email: Mapped[str | None] = mapped_column(String(255), nullable=True)

class Municipality(Base):
    __tablename__ = "municipalities"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    name: Mapped[str] = mapped_column(String(100), unique=True, nullable=False)
    slug: Mapped[str] = mapped_column(String(100), unique=True, index=True, nullable=False)
    center_lat: Mapped[float] = mapped_column(Float, nullable=False)
    center_lng: Mapped[float] = mapped_column(Float, nullable=False)
    boundary_geojson: Mapped[dict | None] = mapped_column(JSONB, nullable=True)

class Place(Base):
    __tablename__ = "places"
    __table_args__ = (
        UniqueConstraint("source_name", "source_record_id", name="uq_place_source_record"),
        Index("ix_places_lat_lng", "latitude", "longitude"),
    )
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    name: Mapped[str] = mapped_column(String(200), index=True, nullable=False)
    category: Mapped[str] = mapped_column(String(64), index=True, nullable=False)
    latitude: Mapped[float] = mapped_column(Float, nullable=False)
    longitude: Mapped[float] = mapped_column(Float, nullable=False)
    municipality_id: Mapped[uuid.UUID | None] = mapped_column(UUID(as_uuid=True),
```

```
ForeignKey("municipalities.id"), index=True)
    address_text: Mapped[str | None] = mapped_column(String(255))
    description: Mapped[str | None] = mapped_column(Text)
    phone: Mapped[str | None] = mapped_column(String(64))
    website: Mapped[str | None] = mapped_column(String(255))
    opening_hours: Mapped[str | None] = mapped_column(String(255))
    estimated_cost_min: Mapped[float | None] = mapped_column(Numeric(10, 2), nullable=True)
    estimated_cost_max: Mapped[float | None] = mapped_column(Numeric(10, 2), nullable=True)
    cost_currency: Mapped[str] = mapped_column(String(3), default="PHP", nullable=False)
    image_url: Mapped[str | None] = mapped_column(String(500))
    image_license: Mapped[str | None] = mapped_column(String(255))
    source_name: Mapped[str] = mapped_column(String(64), default="OpenStreetMap",
nullable=False)
    source_record_id: Mapped[str] = mapped_column(String(128), nullable=False)
    source_url: Mapped[str | None] = mapped_column(String(500))
    imported_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    verified_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True),
nullable=True)
    is_featured: Mapped[bool] = mapped_column(Boolean, default=False, index=True)
    is_active: Mapped[bool] = mapped_column(Boolean, default=True, index=True)
    tags: Mapped[list[str]] = mapped_column(JSONB, default=list)

class Itinerary(Base):
    __tablename__ = "itineraries"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    user_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), ForeignKey("users.id",
ondelete="CASCADE"), index=True, nullable=False)
    title: Mapped[str] = mapped_column(String(160), nullable=False)
    destination: Mapped[str] = mapped_column(String(120), nullable=False)
    start_date: Mapped[date] = mapped_column(Date, nullable=False)
    end_date: Mapped[date] = mapped_column(Date, nullable=False)
    daily_budget: Mapped[float] = mapped_column(Numeric(10, 2), nullable=False)
    preferences: Mapped[list[str]] = mapped_column(JSONB, default=list)
    type: Mapped[str] = mapped_column(String(20), nullable=False)  # 'suggested' | 'custom'
    assumptions: Mapped[list[str]] = mapped_column(JSONB, default=list)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow,
onupdate=utcnow)
    days: Mapped[list["ItineraryDay"]] = relationship(back_populates="itinerary",
cascade="all, delete-orphan", order_by="ItineraryDay.day_number")

class ItineraryDay(Base):
    __tablename__ = "itinerary_days"
    __table_args__ = (UniqueConstraint("itinerary_id", "day_number",
name="uq_itinerary_day_num"),)
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    itinerary_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
ForeignKey("itineraries.id", ondelete="CASCADE"), nullable=False)
    day_number: Mapped[int] = mapped_column(Integer, nullable=False)
    travel_date: Mapped[date] = mapped_column(Date, nullable=False)
    title: Mapped[str | None] = mapped_column(String(120))
    itinerary: Mapped["Itinerary"] = relationship(back_populates="days")
    items: Mapped[list["ItineraryItem"]] = relationship(back_populates="day", cascade="all,
delete-orphan", order_by="ItineraryItem.sort_order")

class ItineraryItem(Base):
    __tablename__ = "itinerary_items"
```

```
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    day_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
ForeignKey("itinerary_days.id", ondelete="CASCADE"), nullable=False)
    place_id: Mapped[uuid.UUID | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("places.id", ondelete="SET NULL"), nullable=True)
    custom_title: Mapped[str | None] = mapped_column(String(200))
    custom_lat: Mapped[float | None] = mapped_column(Float)
    custom_lng: Mapped[float | None] = mapped_column(Float)
    sort_order: Mapped[int] = mapped_column(Integer, default=1, nullable=False)
    planned_time: Mapped[time | None] = mapped_column(Time, nullable=True)
    duration_minutes: Mapped[int] = mapped_column(Integer, default=90, nullable=False)
    notes: Mapped[str | None] = mapped_column(Text)
    estimated_cost_min: Mapped[float | None] = mapped_column(Numeric(10, 2), nullable=True)
    estimated_cost_max: Mapped[float | None] = mapped_column(Numeric(10, 2), nullable=True)
    day: Mapped["ItineraryDay"] = relationship(back_populates="items")
```

3.5 Generate Alembic Migrations & Expose `GET /api/v1/health`

Initialize Alembic, import `Base` from `app.models.entities` inside `alembic/env.py`, and apply the migration:

```
alembic init alembic
# In alembic/env.py, set: target_metadata = Base.metadata
# and config.set_main_option("sqlalchemy.url", settings.database_url)
alembic revision --autogenerate -m "initial_nueva_ecija_schema"
alembic upgrade head
```

Create `backend/app/main.py` with CORS middleware and the health-check route, then write `backend/tests/test_health.py` to verify the endpoint and database session:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.core.config import settings

app = FastAPI(
    title="Nueva Ecija Travel Itinerary API",
    version="0.1.0",
    docs_url="/docs" if settings.app_env == "development" else None,
)
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins,
    allow_credentials=True,
    allow_methods=["GET", "POST", "PATCH", "DELETE", "OPTIONS"],
    allow_headers=["Authorization", "Content-Type"],
)

@app.get(f"{settings.api_v1_prefix}/health")
def health_check():
    return {"status": "ok", "province": "Nueva Ecija"}
```

Step 1 Exit Check: Run `pytest -q` and `uvicorn app.main:app --reload`; confirm `GET http://127.0.0.1:8000/api/v1/health` returns `200 OK` and `\dt` in `psql` lists all 7 tables.

4. Step 2 — Build the Place-Data Import & Curated Quality Pipeline

OpenStreetMap data must be imported offline into PostgreSQL and curated so the app never hammers public Overpass servers at runtime and never presents unverified metadata as confirmed fact.

4.1 Canonical 16 Place Categories & Tag Mapping

Map OpenStreetMap tags to the 16 canonical categories required by Section 6.7 and Section 7.2 of the blueprint:

Canonical Category Key	Map Filter Display Label	OpenStreetMap Source Tag Rules
`cafe`	Cafes	`amenity=cafe`
`restaurant`	Restaurants	`amenity IN (restaurant, fast_food, food_court)`
`hotel_accommodation`	Hotels & Accommodations	`tourism IN (hotel, guest_house, hostel, motel)`
`tourist_attraction`	Tourist Attractions & Landmarks	`tourism IN (attraction, viewpoint)` or `historic IN (monument, memorial)`
`nature_park`	Nature & Parks	`leisure IN (park, nature_reserve)` or `waterway=waterfall`
`resort_swimming`	Resorts & Swimming	`tourism=resort` or `leisure IN (water_park, swimming_pool)`
`church_religious`	Churches & Religious Sites	`amenity=place_of_worship`
`museum_cultural`	Museums & Cultural Locations	`tourism=museum` or `historic=heritage`
`shopping_market`	Shopping & Markets	`shop IN (mall, supermarket)` or `amenity=marketplace`
`transport_terminal`	Transport Terminals	`amenity=bus_station` or `public_transport=station`
`hospital_clinic`	Hospitals & Clinics	`amenity IN (hospital, clinic)`
`pharmacy`	Pharmacies	`amenity=pharmacy`
`bank_atm`	Banks & ATMs	`amenity IN (bank, atm)`
`fuel_station`	Fuel Stations	`amenity=fuel`
`convenience_store`	Convenience Stores	`shop=convenience`
`public_facility`	Public Facilities	`amenity IN (townhall, police, fire_station, community_centre)`

4.2 Export One-Time Overpass Extract & Run Idempotent Import (`scripts/import_osm.py`)

Fetch the initial Nueva Ecija POI dataset once from the terminal into `data/raw/nueva_ecija_osm.json`:

```
curl -G "https://overpass-api.de/api/interpreter" \
  --data-urlencode 'data=[out:json][timeout:90];area["name"="Nueva Ecija"]
["admin_level"="4"]->.ne;(nwr["name"]["amenity"](area.ne);nwr["name"]["tourism"]
(area.ne);nwr["name"]["leisure"](area.ne);nwr["name"]["historic"](area.ne);nwr["name"]
["shop"~"mall|supermarket|convenience"](area.ne););out center tags;' \
  -o data/raw/nueva_ecija_osm.json
```

Create `scripts/import_osm.py` with a `--dry-run` flag, Nueva Ecija bounding-box validation (`15.15 <= lat <= 16.15` and `120.70 <= lng <= 121.40`), and PostgreSQL upsert (`ON CONFLICT ON CONSTRAINT uq_place_source_record DO UPDATE`):

```
import argparse, json
from datetime import datetime, timezone
from sqlalchemy.dialects.postgresql import insert
from app.core.database import SessionLocal
from app.models.entities import Place

NE_BOUNDS = {"min_lat": 15.15, "max_lat": 16.15, "min_lng": 120.70, "max_lng": 121.40}

def is_within_nueva_ecija(lat: float, lng: float) -> bool:
    return NE_BOUNDS["min_lat"] <= lat <= NE_BOUNDS["max_lat"] and NE_BOUNDS["min_lng"] <=
lng <= NE_BOUNDS["max_lng"]

def import_osm_file(filepath: str, dry_run: bool = False) -> int:
    with open(filepath, "r", encoding="utf-8") as f:
        elements = json.load(f).get("elements", [])
    valid_rows = []
    for el in elements:
        tags = el.get("tags", {})
        name = (tags.get("name") or "").strip()
        lat = el.get("lat") or (el.get("center") or {}).get("lat")
        lng = el.get("lon") or (el.get("center") or {}).get("lon")
        category = map_category(tags)  # Maps according to Section 4.1 table
        if not name or lat is None or lng is None or not category or not
is_within_nueva_ecija(float(lat), float(lng)):
            continue
        source_id = f"{el['type']}/{el['id']}"
        valid_rows.append({
            "name": name, "category": category, "latitude": float(lat), "longitude":
float(lng),
            "address_text": tags.get("addr:full") or tags.get("addr:street"),
            "phone": tags.get("phone") or tags.get("contact:phone"),
            "website": tags.get("website") or tags.get("contact:website"),
            "opening_hours": tags.get("opening_hours"),
            "source_name": "OpenStreetMap", "source_record_id": source_id,
            "source_url": f"https://www.openstreetmap.org/{source_id}",
            "imported_at": datetime.now(timezone.utc), "is_active": True, "tags": [category],
        })
    if dry_run:
        print(f"[DRY RUN] Validated {len(valid_rows)} POIs inside Nueva Ecija.")
        return len(valid_rows)
    with SessionLocal() as db:
        for row in valid_rows:
            stmt = insert(Place).values(**row).on_conflict_do_update(
                constraint="uq_place_source_record",
```

```
                set_={"name": row["name"], "category": row["category"], "opening_hours":
row["opening_hours"], "imported_at": row["imported_at"]}
            )
            db.execute(stmt)
        db.commit()
    return len(valid_rows)
```

4.3 Seed Municipalities & Curated Featured Destinations (`scripts/seed_featured_places.py`)

Create `scripts/seed_featured_places.py` to upsert Nueva Ecija municipalities (Cabanatuan City, Gapan City, Science City of Muñoz, Palayan City, San Jose City, General Tinio, Gabaldon, Pantabangan, Bongabon, Laur) and curated `is_featured=True` places with verified descriptions, `verified_at` timestamps, and known cost estimates (leaving unknown costs `None` rather than `0`).

Step 2 Exit Check: Run `python ../scripts/import_osm.py ../data/raw/nueva_ecija_osm.json` twice in succession and assert that the total row count in `places` does not increase on the second run.

5. Step 3 — Build Authentication & the Ionic Angular App Shell

Step 3 wires up password and OAuth security on the backend alongside the Ionic Angular + Capacitor mobile client shell and local guest persistence.

5.1 Backend Authentication & OAuth Token Verification (`backend/app/api/v1/auth.py`)

Implement `backend/app/services/auth_service.py` and `backend/app/api/v1/auth.py` following strict security controls:

```
from pwdlib import PasswordHash
from google.oauth2 import id_token
from google.auth.transport import requests as google_requests

password_hasher = PasswordHash.recommended()  # Uses Argon2id by default

def normalize_identifier(value: str) -> str:
    return value.strip().lower()

def verify_google_token(raw_id_token: str, client_id: str) -> dict:
    idinfo = id_token.verify_oauth2_token(raw_id_token, google_requests.Request(), client_id)
    if idinfo.get("iss") not in ("accounts.google.com", "https://accounts.google.com"):
        raise ValueError("Invalid Google token issuer")
    return idinfo  # Contains stable 'sub', 'email', 'given_name', 'family_name'
```

- **Registration (`POST /api/v1/auth/register`)**: Normalize `username` and `email` to lowercase, check uniqueness, hash the password with Argon2id, and catch SQLAlchemy `IntegrityError` to handle concurrent signup race conditions cleanly with `409 Conflict`.
- **Login (`POST /api/v1/auth/login`)**: Verify password hash and return a signed short-lived JWT access token (30 minutes) plus refresh token. Use a generic `401 Invalid email or password` error so failed logins do not enumerate accounts.
- **Google OAuth (`POST /api/v1/auth/oauth/google`)**: Validate the ID token server-side, look up `AuthIdentity` by `(provider="google", provider_subject=idinfo["sub"])`, and link or create the user account.

- **Conditional Apple OAuth (`POST /api/v1/auth/oauth/apple`)**: Check `settings.enable_apple_auth`; when `False`, return `501 Not Implemented` with a clear explanation.

5.2 Initialize Ionic Angular + Capacitor Project (`mobile/`)

```
cd ..
ionic start mobile tabs --type=angular --capacitor --no-git
cd mobile
npm install leaflet @types/leaflet leaflet.markercluster @types/leaflet.markercluster
@ionic/storage-angular
```

Configure `mobile/src/environments/environment.ts` with replaceable API and tile endpoints (never place secret keys in Angular environment files):

```
export const environment = {
  production: false,
  apiBaseUrl: 'http://127.0.0.1:8000/api/v1',
  tileUrlTemplate: 'https://tile.openstreetmap.org/{z}/{x}/{y}.png',
  tileAttribution: '&copy; <a href="https://www.openstreetmap.org/copyright" target="_blank"
rel="noopener">OpenStreetMap</a> contributors',
  enableAppleAuth: false,
  googleClientId: 'your-google-client-id.apps.googleusercontent.com'
};
```

5.3 Implement Welcome, Login, Signup, Guest Storage & the 4-Tab Shell

- **Welcome, Login & Signup Screens**: Build reactive forms for Signup (`firstName`, `lastName`, `username`, `email`, `password`, `confirmPassword`). Validate matching passwords in an Angular validator and omit `confirmPassword` from the API request body.
- **Truthful Apple Button State**: Bind the Apple login button to `environment.enableAppleAuth`. When `false`, render it disabled with a visible caption: `"Apple Sign-In is disabled in this pilot build"`.
- **Guest Mode Storage Service (`GuestStorageService`)**: Initialize `@ionic/storage-angular` and store guest itineraries under key `ne_guest_itineraries_v1`. Display a non-intrusive notice on Home and Itinerary screens reminding guests that device-local itineraries do not sync across devices unless migrated to an account.
- **Four-Tab Navigation Shell (`tabs.routes.ts`)**: Configure `Home`, `Map`, `History`, and `Profile` tabs. Render a clean placeholder screen on `History` and `Profile` stating `"Planned for a later release — MVP focuses on Home, Map, and Itinerary Planning."`

Step 3 Exit Check: Register a new user, verify case-insensitive duplicate rejection, log in with Email and Google, enter Guest mode, and confirm guest itineraries survive an app reload.

6. Step 4 — Implement the Home Tab & Local Place Search

The Home tab serves as the primary discovery hub, combining local debounced search, Suggested and Custom Itinerary entry cards, Popular Destinations, and Saved Itineraries.

6.1 Backend Search Endpoint (`GET /api/v1/places` & `GET /api/v1/municipalities`)

Implement `backend/app/api/v1/places.py` to filter active Nueva Ecija records by text query, category array, municipality UUID, featured flag, and geographic bounding box:

```
@router.get("/places")
def list_places(
    q: str | None = Query(default=None, max_length=100),
    categories: list[str] | None = Query(default=None),
    municipality_id: uuid.UUID | None = None,
    featured: bool | None = None,
    min_lat: float | None = None, max_lat: float | None = None,
    min_lng: float | None = None, max_lng: float | None = None,
    limit: int = Query(default=50, ge=1, le=200),
    offset: int = Query(default=0, ge=0),
    db: Session = Depends(get_db),
):
    stmt = select(Place).where(Place.is_active.is_(True))
    if q:
        pattern = f"%{q.strip()}%"
        stmt = stmt.where(or_(Place.name.ilike(pattern), Place.address_text.ilike(pattern),
Place.category.ilike(pattern)))
    if categories:
        stmt = stmt.where(Place.category.in_(categories))
    if municipality_id:
        stmt = stmt.where(Place.municipality_id == municipality_id)
    if featured is not None:
        stmt = stmt.where(Place.is_featured.is_(featured))
    if None not in (min_lat, max_lat, min_lng, max_lng):
        stmt = stmt.where(Place.latitude.between(min_lat, max_lat),
Place.longitude.between(min_lng, max_lng))
    total = db.scalar(select(func.count()).select_from(stmt.subquery()))
    items = db.scalars(stmt.order_by(Place.is_featured.desc(),
Place.name.asc()).limit(limit).offset(offset)).all()
    return {"total": total, "items": items}
```

6.2 Frontend Home Tab & RxJS Debounced Search (`mobile/src/app/tabs/home/`)

```
this.searchControl.valueChanges.pipe(
  debounceTime(300),
  distinctUntilChanged(),
  switchMap((query) => {
    const trimmed = (query || '').trim();
    if (!trimmed) { return of({ total: 0, items: [] }); }
    return this.placeService.searchPlaces({ q: trimmed, limit: 20 }).pipe(
      catchError(() => of({ total: 0, items: [], error: true }))
    );
  }),
  takeUntilDestroyed(this.destroyRef)
).subscribe((res) => { this.searchResults = res.items; });
```

- **Popular Destinations Section**: Load `GET /api/v1/places?featured=true&limit=10` and display horizontal or grid cards showing verified licensed imagery (or a clean category icon placeholder when `image_url` is `null`), municipality name, category, and short summary.
- **Saved Itineraries Section**: Subscribe to `ItineraryService.savedItineraries$`, which fetches from `GET /api/v1/itineraries` when logged in or from `GuestStorageService` when in Guest mode.

Step 4 Exit Check: Type rapidly in the Home search bar and confirm in browser DevTools Network tab that stale requests are canceled, Nominatim is never called, and only Nueva Ecija records are returned.

7. Step 5 — Implement the Nueva Ecija Interactive Map Tab

The Map tab renders an interactive Leaflet map bounded to Nueva Ecija, supports multi-select filtering across all 16 categories, displays verified vs. unverified place details in an accessible bottom sheet, and hands off directions externally.

7.1 Leaflet Initialization & Ionic Tab Resize Handling (`map.page.ts`)

Import Leaflet CSS in `angular.json`, give `#ne-map` an explicit height (`height: calc(100vh - 112px); width: 100%;`), and always invoke `this.map.invalidateSize()` inside Ionic's `ionViewDidEnter()` lifecycle hook:

```
const NE_CENTER: L.LatLngExpression = [15.58, 121.00];
const NE_MAX_BOUNDS = L.latLngBounds([15.10, 120.65], [16.20, 121.45]);

ionViewDidEnter(): void {
  if (!this.map) {
    this.initMap();
  } else {
    setTimeout(() => this.map.invalidateSize(), 150);
  }
}

private initMap(): void {
  this.map = L.map('ne-map', {
    center: NE_CENTER,
    zoom: 10,
    minZoom: 9,
    maxZoom: 18,
    maxBounds: NE_MAX_BOUNDS,
    maxBoundsViscosity: 0.9,
  });

  const tileLayer = L.tileLayer(environment.tileUrlTemplate, {
    attribution: environment.tileAttribution,
    maxZoom: 18,
  });

  tileLayer.on('tileerror', () => {
    this.tileWarningVisible = true; // Shows non-blocking banner; keeps itineraries & list
usable
  });

  tileLayer.addTo(this.map);
  this.map.on('moveend', () => this.loadVisiblePlaces());
  this.loadVisiblePlaces();
}
```

7.2 Multi-Select Category Filters, Marker Detail Sheet & External Directions

- **Filter Control Modal**: Provide a visible Filter button with multi-select checkboxes for all 16 categories, plus **Reset** and **Apply** buttons. Combining category filters with search or map bounds queries `GET /api/v1/places` and refreshes markers via `leaflet.markercluster`.
- **Accessible List View Toggle**: Add a segmented toggle (`Map View | List View`) so travelers on small screens or assistive technologies can browse and select the filtered POIs without precision pin tapping.

- **Truthful Marker Bottom Sheet**: When a marker is tapped, open an `ion-modal` bottom sheet displaying:
  - Place Name, Category label, and Municipality / Address.
  - Opening Hours: `{{ selectedPlace.opening_hours || 'Not verified / Not provided' }}`.
  - Estimated Cost: `{{ formatCostRange(selectedPlace) || 'Cost not provided' }}`.
  - Verified Description / Facts: `{{ selectedPlace.description || 'Detailed description not yet verified.' }}`.
  - Provenance Footer: `Source: {{ selectedPlace.source_name }} | Verified: {{ (selectedPlace.verified_at | date:'mediumDate') || 'Unverified import' }}`.
- **External Directions Hand-Off**: Bind the **Get Directions** button to open a key-free Google Maps URL in the system browser/maps app:

```
openDirections(place: Place): void {
  const url = `https://www.google.com/maps/dir/?
api=1&destination=${place.latitude},${place.longitude}`;
  window.open(url, '_blank', 'noopener');
}
```

Step 5 Exit Check: Verify that `© OpenStreetMap contributors` is visible, all 16 filters work, missing fields display `"Not verified / Not provided"`, and simulated tile failures do not break saved itineraries.

8. Step 6 — Build the Custom Itinerary Flow & Ownership-Checked API

Travelers must be able to author multi-day custom itineraries, mix database POIs with personal custom stops, reorder schedules, view honest cost totals, and never lose unsaved edits when switching tabs.

8.1 BOLA-Safe Itinerary CRUD Endpoints (`backend/app/api/v1/itineraries.py`)

Enforce ownership verification on every single-itinerary and stop-level operation so one authenticated user can never read, edit, or delete another user's itinerary by guessing a UUID:

```
def get_owned_itinerary(db: Session, itinerary_id: uuid.UUID, current_user_id: uuid.UUID) ->
Itinerary:
    itin = db.scalar(
        select(Itinerary).where(Itinerary.id == itinerary_id, Itinerary.user_id ==
current_user_id)
    )
    if not itin:
        raise HTTPException(status_code=404, detail="Itinerary not found")
    return itin

@router.post("/itineraries/{id}/reorder")
def reorder_itinerary_items(
    id: uuid.UUID,
    payload: ReorderPayload,  # Map of day_id -> ordered list of item_ids
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db),
):
    itin = get_owned_itinerary(db, id, current_user.id)
    valid_day_ids = {d.id for d in itin.days}
    for day_id, ordered_item_ids in payload.day_orders.items():
        if day_id not in valid_day_ids:
```

```
            raise HTTPException(status_code=400, detail="Invalid day reference for this
itinerary")
        for idx, item_id in enumerate(ordered_item_ids, start=1):
            db.execute(
                update(ItineraryItem)
                .where(ItineraryItem.id == item_id, ItineraryItem.day_id == day_id)
                .values(sort_order=idx)
            )
    db.commit()
    return get_owned_itinerary(db, id, current_user.id)
```

8.2 Frontend Custom Builder, Honest Cost Math & Unsaved-Changes Guard

- **Adding Database or Custom Stops**: Allow adding stops either from the local Nueva Ecija place picker or via a manual custom-stop dialog (`custom_title`, optional `custom_lat`/`custom_lng`, `planned_time`, `duration_minutes`, `notes`). If coordinates are omitted on a custom stop, display it in the day list with a note `"No map pin (coordinates not provided)"`.
- **Same-Day Duplicate Stop Guard**: When adding a `place_id` to a day that already contains that `place_id`, present an `IonAlert` confirmation before inserting.
- **Honest Budget Calculation Helper**: Never treat `null` cost as `0`:

```
calculateDayCostSummary(items: ItineraryItem[]): { minKnown: number; maxKnown: number;
unknownCount: number } {
  let minKnown = 0, maxKnown = 0, unknownCount = 0;
  for (const item of items) {
    if (item.estimated_cost_min != null && item.estimated_cost_max != null) {
      minKnown += Number(item.estimated_cost_min);
      maxKnown += Number(item.estimated_cost_max);
    } else {
      unknownCount += 1;
    }
  }
  return { minKnown, maxKnown, unknownCount };
}
```

- **Unsaved Changes Route Guard (`pendingChangesGuard`)**: Attach a `CanDeactivateFn` guard to the itinerary editor route that prompts `"You have unsaved itinerary changes. Discard changes and leave?"` if `hasUnsavedChanges === true`.

Step 6 Exit Check: Create a multi-day custom itinerary, reorder items across days, verify that another user receives `404` when attempting to PATCH or DELETE it, and confirm unsaved-change prompts work on tab switches.

9. Step 7 — Build the Deterministic Suggested Itinerary Engine

Instead of calling a paid or hallucination-prone generative AI API, implement the configurable, deterministic scoring and scheduling service specified in Section 8 of the blueprint.

9.1 Configurable Scoring Formula (`backend/app/algorithms/itinerary_generator.py`)

```
import math
from dataclasses import dataclass
from datetime import date, time, timedelta
from app.models.entities import Place
```

```
@dataclass(frozen=True)
class ScoringConfig:
    w_preference: float = 40.0
    w_proximity: float = 30.0
    w_featured: float = 15.0
    w_diversity_penalty: float = 20.0
    w_cost_fit: float = 10.0
    max_radius_km: float = 25.0
    stops_per_day: int = 4
    default_duration_mins: int = 90

PREFERENCE_CATEGORY_MAP: dict[str, set[str]] = {
    "Nature & Parks": {"nature_park"},
    "Food Trip": {"restaurant", "shopping_market"},
    "Culture & History": {"museum_cultural", "tourist_attraction"},
    "Shopping": {"shopping_market"},
    "Resort/Swimming": {"resort_swimming"},
    "Cafes": {"cafe"},
    "Family-Friendly": {"nature_park", "resort_swimming", "museum_cultural",
"tourist_attraction"},
    "Religious/Cultural Sites": {"church_religious", "museum_cultural"},
}

def haversine_km(lat1: float, lng1: float, lat2: float, lng2: float) -> float:
    r = 6371.0
    dlat, dlng = math.radians(lat2 - lat1), math.radians(lng2 - lng1)
    a = math.sin(dlat / 2) ** 2 + math.cos(math.radians(lat1)) * math.cos(math.radians(lat2))
* math.sin(dlng / 2) ** 2
    return 2 * r * math.asin(math.sqrt(a))

def score_poi(
    place: Place, cluster_lat: float, cluster_lng: float,
    preferred_cats: set[str], remaining_budget: float,
    prev_category: str | None, cfg: ScoringConfig = ScoringConfig()
) -> float:
    dist_km = haversine_km(cluster_lat, cluster_lng, place.latitude, place.longitude)
    if dist_km > cfg.max_radius_km:
        return -1.0
    score = 0.0
    if place.category in preferred_cats or any(t in preferred_cats for t in (place.tags or
[])):
        score += cfg.w_preference
    score += cfg.w_proximity * max(0.0, 1.0 - (dist_km / cfg.max_radius_km))
    if place.is_featured or place.verified_at is not None:
        score += cfg.w_featured
    if prev_category and place.category == prev_category:
        score -= cfg.w_diversity_penalty
    if place.estimated_cost_max is not None:
        score += cfg.w_cost_fit if float(place.estimated_cost_max) <= remaining_budget else -
cfg.w_cost_fit
    return round(score, 2)
```

9.2 Daily Clustering, Transparency Assumptions & Sparse-Data Handling

1. **Input Validation**: Reject requests where `end_date < start_date`, duration exceeds 14 days, or `daily_budget <= 0` with `422 Unprocessable Content`.

2. **Geographic Clustering & Time Slots**: Anchor Day 1 at the requested Nueva Ecija municipality center. For each subsequent stop on the same day, update `(cluster_lat, cluster_lng)` to the coordinates of the just-scheduled stop so nearby POIs are grouped together without backtracking. Assign default time slots (`09:00`, `11:30`, `14:00`, `16:30`) and `90`-minute estimated durations.
3. **Explicit Assumptions Banner**: Return an `assumptions` array alongside the generated draft:
   - `"Proximity grouping uses straight-line Haversine distance, not live road traffic or transit schedules."`
   - `"Default 90-minute visit durations are estimates where venue duration is unverified."`
   - `"Unknown admission or meal costs are labeled 'Cost not provided' and excluded from numeric spend totals."`
4. **Sparse-Data Fallback**: If fewer than 3 candidate POIs exist within `max_radius_km` for a day, schedule only the verified available places and append: `"Fewer stops were scheduled on Day X because verified POI coverage near this municipality is currently limited."` Never fabricate placeholder venues.

Step 7 Exit Check: Write unit tests in `backend/tests/test_itinerary_generator.py` verifying preference weighting, category diversity penalty, sparse-data fallback, and non-zero handling of unknown prices.

10. Step 8 — Connect Itinerary Overview, Day Details & Home Persistence

In Step 8, unify the Suggested and Custom flows so accepted drafts and custom schedules share the same Overview, Day Detail, and Home card components.

- **Itinerary Overview Screen**: Display the itinerary title, Nueva Ecija destination badge, date range, day count, daily PHP budget, selected preference chips, the algorithm assumptions/transparency box, and interactive day-selector chips (`Day 1`, `Day 2`, ...).
- **Day Detail & Stop Detail Cards**: List ordered stops for the selected day with scheduled time, estimated duration, category, known cost range (or `"Cost not provided"`), notes, and inline controls to **Edit**, **Reorder**, **Delete**, or open external **Get Directions**.
- **Reactive Home Tab Synchronization**: Whenever an itinerary is saved, updated, or deleted (in PostgreSQL for signed-in users or in `IonicStorage` for guests), emit a refresh event in `ItineraryService` so Home tab itinerary cards update immediately upon returning to the Home tab.

Step 8 Exit Check: Generate a Suggested Itinerary, customize two stops, save it, verify it appears on the Home tab, edit Day 2, and confirm changes persist after restarting the app.

11. Step 9 — Secure the Laptop-Hosted API, Cross-Device Access & Backups

Prepare your laptop-hosted backend for multi-device testing on local Wi-Fi or a remote pilot via Cloudflare Tunnel while keeping PostgreSQL isolated and recoverable.

11.1 Local Wi-Fi LAN Testing vs. Remote Cloudflare Tunnel Pilot

Test Mode	Exact Setup Commands	Security & Network Rules
Same Wi-Fi LAN	`uvicorn app.main:app --host 0.0.0.0 --port 8000` Set `apiBaseUrl` to `http://<LAPTOP_LAN_IP>:8000/api/v1` and run `npx cap sync`.	Keep PostgreSQL bound to `127.0.0.1:5432` in `postgresql.conf`. Allow inbound TCP `8000` only on the local private subnet.
Remote Pilot (HTTPS Tunnel)	`uvicorn app.main:app --host 127.0.0.1 --port 8000` `cloudflared tunnel --url http://127.0.0.1:8000`	No router port-forwarding required. Add the HTTPS tunnel origin to `CORS_ORIGINS` and set `APP_ENV=pilot` to disable `/docs`.

11.2 Rate Limiting, Offline Laptop Banner & Database Backup Verification

- **Authentication Rate Limiting**: Configure `slowapi` on `/api/v1/auth/login` and `/api/v1/auth/register` (e.g., `5/minute` per IP) to mitigate credential stuffing.
- **Graceful Laptop-Offline State**: When the laptop sleeps or the tunnel disconnects, intercept `0` / `502` / `504` HTTP errors in an Angular HTTP interceptor and show a clear banner: `"Pilot backend is currently unreachable (host laptop may be offline). Local guest itineraries remain accessible."`
- **PostgreSQL Backup & Restore Drill**: Run and verify a full backup/restore cycle before starting pilot testing:

```
pg_dump -U ne_travel_user -h 127.0.0.1 -Fc nueva_ecija_travel > ne_travel_pilot.dump
createdb -U postgres -h 127.0.0.1 -O ne_travel_user nueva_ecija_travel_verify
pg_restore -U ne_travel_user -h 127.0.0.1 -d nueva_ecija_travel_verify
ne_travel_pilot.dump
```

Step 9 Exit Check: Access the app from a physical Android/iOS device over LAN or Cloudflare Tunnel, verify port `5432` is unreachable from the second device, and confirm the backup restores cleanly.

12. Step 10 — Automated Verification Suite & MVP Acceptance Checklist

Before declaring the MVP ready for pilot users, run the automated verification commands below and walk through the final QA checklist from Section 13 of the blueprint.

12.1 Command-Line Verification Matrix

Component	Verification Command	Passing Criterion
Alembic Schema	`cd backend && alembic downgrade base && alembic upgrade head`	Clean migration down and up on fresh DB.
POI Import Idempotency	`cd backend && python ../scripts/import_osm.py ../data/raw/nueva_ecija_osm.json`	Re-running creates zero duplicate POI rows.

Component	Verification Command	Passing Criterion
Backend Lint & Types	`cd backend && ruff check . && mypy app`	Zero lint warnings or type mismatches.
Backend Pytest Suite	`cd backend && pytest -q`	100% pass across Auth, Places, BOLA & Suggest tests.
Mobile Lint & Build	`cd mobile && npm run lint && npm test -- --watch=false && ionic build`	Clean Angular unit test pass and production build.

12.2 Final 24-Point MVP Acceptance Gate

- **Auth & Guest Mode (7 checks)**: (1) Valid signup succeeds; (2) Case-insensitive duplicate username/email is rejected; (3) Malformed/wrong credentials show safe errors; (4) Passwords use Argon2id and never appear in logs; (5) Google ID tokens are verified server-side; (6) Guest itineraries persist locally across restarts; (7) Apple login button truthfully reflects its disabled/configured flag.
- **Home & Places (3 checks)**: (8) Search returns only Nueva Ecija records with 300ms debounce and zero Nominatim calls; (9) Curated Popular Destinations open verified detail pages; (10) Null images, costs, or opening hours render graceful fallbacks without layout breakage.
- **Map Tab (7 checks)**: (11) Map opens centered and bounded on Nueva Ecija; (12) `© OpenStreetMap contributors` attribution is visible; (13) All 16 category filters work individually and in combination; (14) Tapping a marker opens the matching POI detail card; (15) Unknown hours/facts are labeled `"Not verified / Not provided"`; (16) Tile/API errors trigger a recoverable state; (17) External Google Maps URL directions open the exact destination coordinates.
- **Itineraries & Operations (7 checks)**: (18) Invalid date ranges and non-positive budgets are rejected; (19) Suggested itineraries respect Nueva Ecija bounds and preferences; (20) Unknown costs are never summed as zero; (21) Custom stops can be added, edited, reordered, and deleted; (22) Cross-user IDOR attempts return `404`; (23) Edited itineraries update immediately on Home; (24) PostgreSQL port `5432` remains closed externally and database backup restoration succeeds.